

Asistente Inteligente de Validación de Cobertura — Documento de Análisis

Prueba Técnica: Ingeniero de IA / Desarrollador de Sistemas Agénticos y RAG (V 2.0) · Julio de 2026

1. Resumen ejecutivo

La solución es un asistente RAG que, dada una consulta formal y el identificador del afiliado, recupera evidencia de los tres documentos del plan, la cruza con el registro estructurado del afiliado, razona sobre la procedencia de la cobertura y redacta una respuesta formal, trazable y citada. Se construyó sobre **Clean Architecture** (FastAPI + ChromaDB + LLM intercambiable por configuración) con una regla central: **el LLM nunca es la fuente de verdad** — solo interpreta la evidencia recuperada y los hechos verificados del afiliado.

118 pruebas automatizadas

96% cobertura de código

10/10 golden set con LLM real

0 errores MyPy strict / Ruff

3 interfaces: Web UI, Swagger, CLI

2. Arquitectura y flujo de razonamiento

Presentation (FastAPI: UI web, /query, /reindex, /health, /metrics)
→ Application (AnalyzeCoverageUseCase · ReasoningEngine · PromptBuilder v1.0 · Validator)
→ Domain (entidades, reglas determinísticas, contratos/ports) ← Infrastructure
(OpenAI/Gemini/Ollama/mock · Chroma · Excel · Word)

Flujo de una consulta (una sola llamada al LLM):

```
{affiliate_id, question}
→ Afiliado (Excel) → Reglas determinísticas (estado, mora, vigencia de autorización)
→ Retrieval vectorial con umbral de relevancia + deduplicación de contexto
→ Evaluación de evidencia (INSUFFICIENT | PARTIAL | STRONG, por reglas explícitas)
→ [sin evidencia ⇒ respuesta honesta SIN invocar el LLM]
→ Prompt modular versionado (guardrails + hechos verificados + evidencia citable)
→ LLM (1 llamada, JSON) → Validación determinística (citas ⊆ chunks entregados)
→ Respuesta estructurada {status, summary, reasoning, evidence[], warnings, trace}
```

Decisiones de diseño principales

Decisión	Justificación
Identificador de afiliado estructurado en el request (nunca extraído del texto)	Los nombres no son únicos y son dato sensible; el enunciado entrega el ID como parte de la entrada.
Reglas determinísticas en código (estado, mora, vigencia); reglas documentales al LLM	Lo verificable no se delega a IA; los umbrales de carencia viven en los documentos, que son la fuente de verdad — no se duplican en código.
Citas verificables como guardrail: se rechaza toda cita a un fragmento no suministrado	Anti-alucinación determinística; toda decisión exige ≥ 1 cita.

<code>EvidenceStrength</code> por reglas explícitas en lugar de "confidence" numérico	Los LLM no producen confianza calibrada; un número inventado sería falsa precisión.
Proveedor LLM/embeddings intercambiable por configuración (OpenAI, Gemini, Ollama, mock)	Ports & Adapters + factories; verificado en la práctica: la solución se evaluó con Gemini gratuito sin tocar código.
Índice con manifiesto (modelo de embeddings + parámetros + hash por documento)	Indexación idempotente e incremental; cambiar de proveedor invalida y reconstruye el índice automáticamente.
Datos personales nunca viajan al prompt ni a logs (filtro de redacción + prueba dedicada)	Datos de salud; privacidad por diseño.

Herramientas elegidas

Herramienta	Rol	Por qué
Python 3.12 + uv	Lenguaje y gestión	Ecosistema IA estándar; reproducibilidad con lockfile.
FastAPI + Pydantic v2	API y contratos	Tipado, validación automática, OpenAPI generado.
ChromaDB (persistente local)	Búsqueda vectorial	Cero infraestructura para el volumen de la prueba; sustituible vía port <code>VectorStore</code> .
GPT-4.1 / Gemini / Ollama vía protocolo OpenAI	Razonamiento y redacción	Un solo adapter cubre múltiples proveedores; <code>OPENAI_BASE_URL</code> lo selecciona.
python-docx con extracción de tablas en orden	Ingesta	Las tablas de DOC1/DOC2 contienen copagos y carencias: un extractor solo de párrafos perdería la información clave.
pytest + MyPy strict + Ruff	Calidad	Gates automáticos; pruebas 100% offline (dobles inyectados).

3. Esquema de evaluación autónoma

Se diseñó en dos ejes, como exige la prueba:

Eje A — Calidad de las respuestas (el modelo)

- **Golden set de 10 casos** (`tests/fixtures/golden_set.json` + `scripts/evaluate_golden_set.py`) diseñado sobre perfiles reales de la BD para cubrir los tres escenarios del enunciado y los bordes: autorización vencida, retirado en mora, beneficiario, exclusión estética, pregunta fuera de dominio, copagos en tablas, afiliado inexistente. Cada caso declara los status aceptables y si debe existir evidencia citada. Métricas: exactitud de status, presencia y validez de citas, reconocimiento de incertidumbre.
- **Guardrails medibles**: el validador rechaza citas a fragmentos no suministrados (alucinación detectable) y decisiones sin evidencia; hay caso explícito fuera de dominio que debe responder "información insuficiente" con cero citas.
- **Interpretación**: el golden set es la herramienta de regresión para cambios de prompt (versionado v1.0) y de parámetros de retrieval.

Eje B — Robustez de la implementación (el software)

- **118 pruebas** en pirámide (67 unitarias / 37 integración / 14 E2E), 100% offline mediante dobles inyectados (nunca OpenAI/Gemini real en CI). Cobertura 96%.
- **Casos límite**: documento corrupto/vacío no detiene la ingesta; índice vacío responde honesto sin invocar el LLM; proveedor LLM caído → 502 tipificado; consulta de 9.000 caracteres; app sin API Key arranca en modo degradado.
- **Garantías arquitectónicas ejecutables**: prueba AST de pureza del dominio + reglas de imports por capa en el linter.
- **Observabilidad**: traza JSON por consulta (`trace_id`, `chunks`, citas, modelo, versión de prompt, latencia) y endpoint `/metrics`.

4. Resultados

Verificación	Resultado
Suite automatizada (unit + integración + E2E)	118/118 · cobertura 96% · MyPy strict y Ruff sin errores
Golden set con LLM real (gemini-3.5-flash + gemini-embedding-001)	10/10 casos aceptados, todos con evidencia STRONG y citas válidas
Consulta formal literal del enunciado (A-00427)	Detectó carencia de 6 meses vs 3 reales, contrastó lo afirmado por el afiliado contra el registro, citó los 3 documentos
Afiliado retirado en mora (A-00003, cirugía electiva)	Rechazo fundamentado citando DOC1+DOC2+DOC3 (similitudes 0.71–0.74)
Pregunta fuera de dominio ("clima en Bogotá")	Información insuficiente, cero citas — no inventó
Cambio de proveedor sin tocar código	mock → Gemini solo con variables de entorno; el manifiesto reconstruyó el índice automáticamente (256 → 3072 dimensiones)

5. Hallazgos y mejoras identificadas

- **Taxonomía de requisitos subsanables:** DOC2 §7 establece que carencia o autorización vencida (subsanables) deben concluir "cubierto con condiciones", pero en algunos casos el sistema emite "no cubierto" cuando el fragmento del orden de verificación no entra al top-k del retrieval. Mejora concreta: subir `TOP_K_RESULTS` (6–8) y/o reforzar la instrucción de razonamiento; el golden set ya acepta ambos status para medir el efecto de la calibración.
- **Umbral de relevancia y de EvidenceStrength** definidos con criterio inicial; son configurables y su calibración fina requiere más consultas reales.
- **Integración real como fuente de hallazgos:** probar con Gemini destapó dos diferencias de proveedor resueltas en el adapter (embeddings sin campo `index`; dimensionalidad de la colección al cambiar de modelo) — evidencia práctica del valor del desacoplamiento.
- **Evolución prevista:** validación semántica con LLM-juez, re-ranking, historial de conversación y repositorio de afiliados en PostgreSQL — todos tienen puerto de entrada definido sin rediseño.

6. Conclusiones

La solución cumple el objetivo funcional de la prueba con énfasis en lo que el enunciado declara valorar: razonamiento trazable (toda decisión reconstruible desde sus citas y su traza), calidad de código verificable con gates automáticos, arquitectura desacoplada demostrada en la práctica (el mismo sistema corrió con mock, y con Gemini gratuito, sin cambiar una línea de código), y un esquema de evaluación propio con resultados medibles. Las limitaciones conocidas están documentadas con su plan de mejora, priorizando profundidad y honestidad técnica sobre exhaustividad.

Ejecución y prueba: ver README.md (instalación con uv, indexación, API con interfaz web en `/`, Swagger en `/docs`, golden set por CLI, Docker). Documentación complementaria en el repositorio: ARCHITECTURE.md, ARCHITECTURE_REVIEW.md, ROADMAP.md, CHANGELOG.md y revisiones por fase (REVIEW_PHASE_2..8).